Abril 10, 2026

Patrones de Software

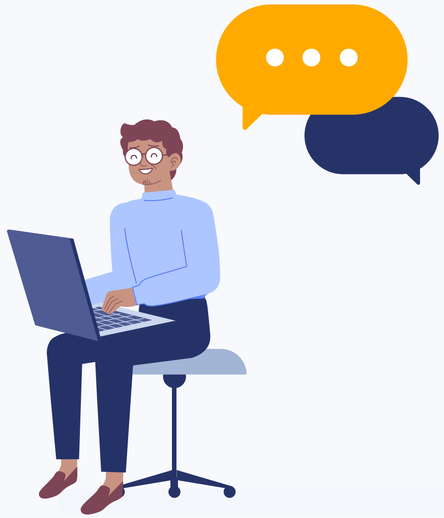
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Recordemos

Conceptos vistos en la anterior clase



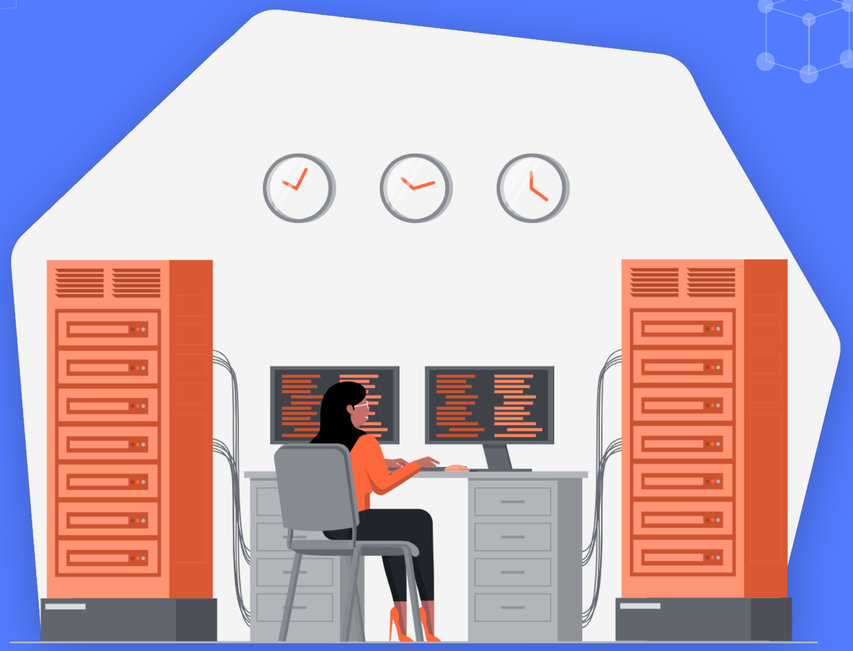
Patrón Decorator

Ventajas, desventajas De Decorator

Practica Decorator

Patrones de Software

Patrón Facade



Definiciones

Es un patrón estructural que proporciona una interfaz simplificada a un conjunto más grande y complejo de clases.

El **patrón Facade** actúa como una fachada, ocultando la complejidad interna y proporcionando una interfaz única y simplificada para interactuar con el sistema subyacente.

Estructura: Patrón Facade

- **Fachada (Facade)**: Proporciona una interfaz unificada y simplificada para el cliente. Es responsable de interactuar con los diferentes sub-sistemas internos para realizar las operaciones requeridas.

- **Sub-sistemas**: Son los módulos o clases individuales que realizan tareas específicas dentro del sistema. Estos sub-sistemas no conocen la existencia del otro y están encapsulados.

Ventajas: Patrón Facade

- **Simplifica la interfaz**: proporciona una interfaz simple y unificada, ocultando la complejidad subyacente. Esto facilita el uso del sistema para los clientes.

- **Desacopla el cliente de la implementación**: El cliente no necesita conocer los detalles internos de cómo se realizan las operaciones. Esto permite cambios en la implementación sin afectar al cliente.

- **Mejora la legibilidad del código**: Al proporcionar una interfaz simplificada, el código cliente se vuelve más fácil de entender y mantener.

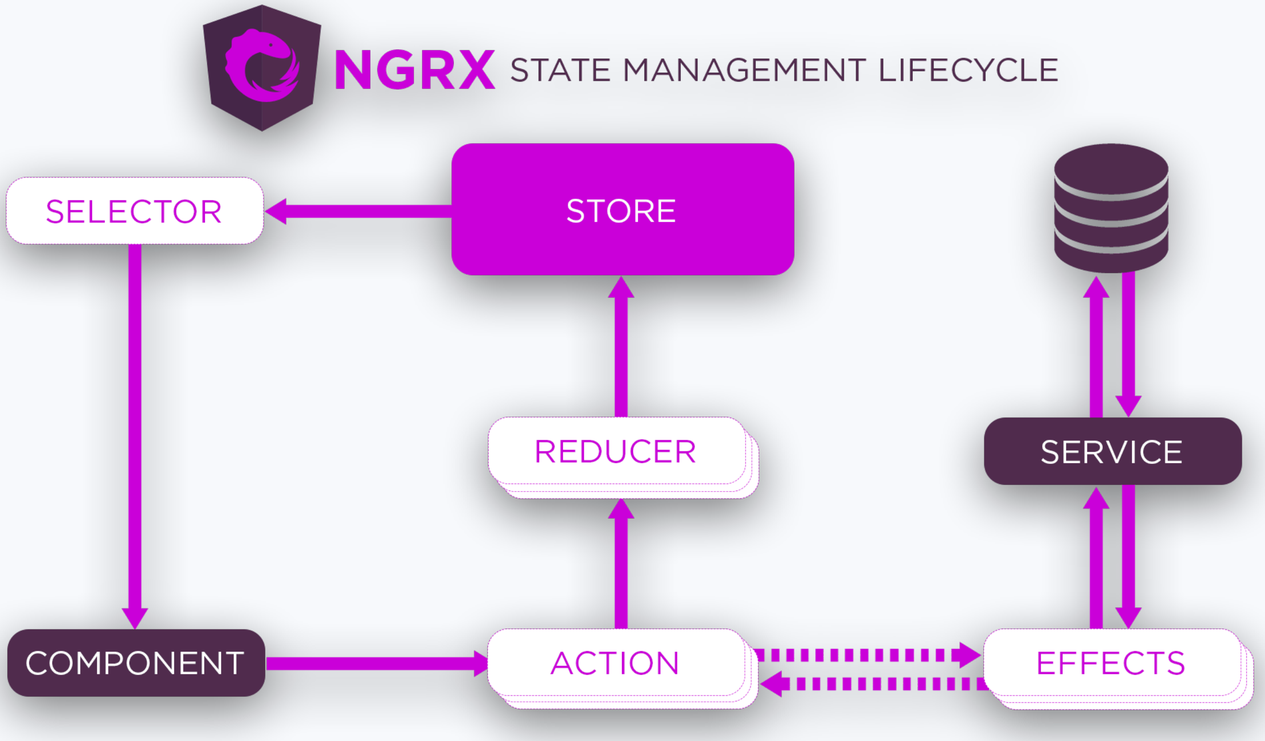
Desventajas: Patrón Facade

- **Menos flexibilidad**: La fachada puede limitar la flexibilidad porque los clientes están restringidos a la interfaz proporcionada por la fachada. Si los clientes necesitan acceder a funciones más avanzadas o específicas, pueden encontrar limitaciones.

- **Aumenta la dependencia**: Los clientes que utilizan la fachada pueden volverse altamente dependientes de ella. Si la fachada cambia, puede afectar a muchos clientes, lo que puede ser problemático.

- **Complejidad interna no eliminada**: Aunque la fachada oculta la complejidad al cliente, no la elimina. La complejidad sigue existiendo internamente, y cualquier cambio en esta complejidad puede requerir modificaciones en la fachada.

Ejemplo: Patrón Facade



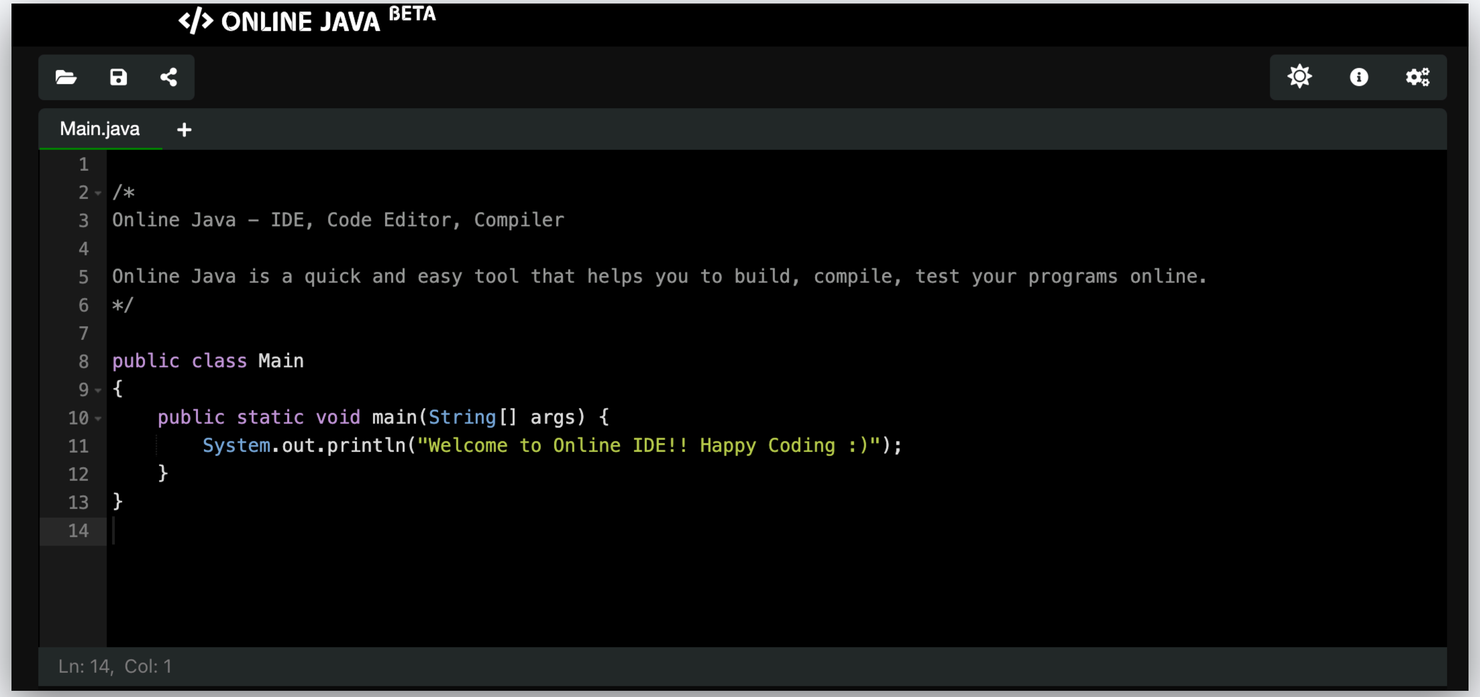
Patrones de Software

Practica del Patrón Facade

Recursos IDE

Online-java

https://www.online-java.com/



Código de la clase

(Tienda en linea)

```java
// Inventario
class Inventory {
    public void checkInventory(String product) {
        System.out.println("Checking inventory for " + product);
    }
}
```

Código de la clase

(Tienda en linea)

```java
// Pagos
class Payments {
    public void processPayment(String product, int quantity, String card) {
        System.out.println("Processing payment for " + quantity +
            " units of " + product + " with card " + card);
    }
}
```

Código de la clase

(Tienda en linea)

```
// Envios
class Shipping {
    public void shipProduct(String product, String address) {
        System.out.println("Shipping " + product +
            " to address " + address);
    }
}
```

Código de la clase

(Tienda en linea)

```java
// Facade
class OnlineStoreFacade {
    private Inventory inventory;
    private Payments payments;
    private Shipping shipping;

    public OnlineStoreFacade() {
        this.inventory = new Inventory();
        this.payments = new Payments();
        this.shipping = new Shipping();
    }

    // Method that provides a unified interface for making a purchase
    public void makePurchase(String product,
        int quantity, String card, String address) {

        inventory.checkInventory(product);
        payments.processPayment(product, quantity, card);
        shipping.shipProduct(product, address);
        System.out.println("Purchase completed successfully");
    }
}
```

Código de la clase

(Tienda en linea)

```java
// Client
public class Main {
    public static void main(String[] args) {
        // Using the Facade to make a simplified purchase
        OnlineStoreFacade storeFacade = new OnlineStoreFacade();
        storeFacade.makePurchase("Laptop", 2,
            "1234-5678-9101-1121", "123 Main Street");
    }
}
```

Patrones de Software

Taller Patrón Facade

TALLER

CASO DE ESTUDIO

- **Nombre Proyecto**: Sistema de Gestión Médica.

- **Contexto**: Una clínica necesita un portal para pacientes donde puedan agendar citas, ver su historia clínica, recibir prescripciones médicas y solicitar exámenes de laboratorio. Internamente, cada una de estas operaciones involucra múltiples validaciones y servicios especializados que el paciente no debe conocer.

- **NOTA 1**: Para este taller se debe crear Frontend y Backend. Para el Backend utilizar Sprint Boot y para el Frontend utilizar cualquier librería de Javascript.
- **NOTA 2**: Tanto el Frontend como el Backend deben estar desplegados en PROD.
- **NOTA 3**: El taller se puede hacer en grupos máximo de 3 personas.

TALLER

Requerimientos del Backend

- **Subsistemas requeridos**: (1) PacienteService: registro con datos personales, validación de documento único, consulta de perfil. (2) AgendaService: disponibilidad de médicos por especialidad (mínimo 3 especialidades, 5 médicos predefinidos), agendar y cancelar citas, recordatorios en la respuesta. (3) HistoriaClinicaService: registrar y consultar consultas previas, diagnósticos y alergias del paciente. (4) PrescripcionService: generar prescripción con medicamentos, dosis y duración. (5) LaboratorioService: solicitar exámenes (hemograma, glicemia, perfil lipídico), retornar resultados simulados con valores de referencia.

- **ClinicaFacade (obligatorio)**: Métodos: agendarCita(pacienteId, especialidad, fecha), verHistoriaCompleta(pacienteId) que consolide citas pasadas + prescripciones + resultados de laboratorio en un solo objeto de respuesta, generarPrescripcion(pacienteId, medicamentos[]) con validación de alergias, solicitarExamenes(pacienteId, examenes[]). Cada método debe invocar mínimo 2 subsistemas.

- **Endpoints REST**: POST /api/clinica/paciente, POST /api/clinica/cita, GET /api/clinica/historia/{pacienteId}, POST /api/clinica/prescripcion, POST /api/clinica/laboratorio, GET /api/clinica/medicos?especialidad=. Datos completamente en memoria.

TALLER

Requerimientos del Frontend

- Registro e inicio de sesión.
- Dashboard con próximas citas y últimos resultados.
- Módulo para agendar nueva cita con selección de especialidad, médico y horario disponible.
- Historia clínica completa con pestañas: consultas, prescripciones, laboratorios.
- Visualización de resultados de laboratorio con indicador visual de si el valor está dentro o fuera del rango normal.

TALLER

Requerimientos del Despliegue

- **Backend** en Railway u otro que maneje con experiencia.
- **Frontend** en Vercel.